

SANCHARi — Slide Content Reference

Satellite Road Extraction Pipeline | Presentation Content Document

This document is a comprehensive reference for building presentation slides on the SANCHARi model. It contains detailed technical summaries, evolution context, script-level explanations, module breakdowns, and ready-to-use slide section content. Illustrations are called out as placeholders where needed.

1. PROJECT OVERVIEW

What is SANCHARi?

SANCHARi (सञ्चारी — Sanskrit for "one who travels") is an end-to-end open-source deep-learning pipeline for extracting road networks from satellite imagery. It accepts coordinates → fetches or reads satellite imagery → runs inference → and returns a georeferenced GeoJSON of the road network. The pipeline is deployable via two FastAPI servers or as standalone batch scripts, and is fully Dockerized.

Core Mission: Democratize road mapping using freely available satellite imagery (NAIP, Sentinel) for use cases like disaster response, infrastructure planning, and region-level GIS where vector maps are absent or outdated.

Current Status (V4):

- Architecture: U-Net++ with EfficientNet-B4 encoder
- Training Dataset: DeepGlobe Road Extraction (Kaggle)
- Accuracy: ~80% IoU on validation set (targeting 85%+)
- Output: GeoJSON FeatureCollections of polyline road networks
- Deployment: Docker (GPU-optional), FastAPI on ports 8000 (local) and 8001 (GEE)

2. ARCHITECTURE EVOLUTION — V1 → V4

2.1 Version Comparison Table

Feature	V1 (Baseline)	V2 (Transfer Learning)	V3 (Refinement)	V4 (Current — State-of-the-Art)
Backbone	Custom U-Net (scratch)	ResNet34 encoder	ResNet34 + Attention gates	EfficientNet-B4 (timm)
Decoder	Basic U-Net	U-Net	U-Net w/ Attention	U-Net++ (Nested)
Input Patch	256×256	256×256	1024×1024 sliding	512×512, 50% overlap
Loss Function	BCE Loss	Dice Loss	Dice Loss	Combo (Dice+Focal) + Lovász
Augmentation	Minimal flips	Standard	Standard	GridDistortion + ElasticTransform
Inference Strategy	Direct patch	Direct patch	4-Way TTA	4-Way TTA + Sliding Window
Post-Processing	Threshold only	Threshold only	Basic morphology	Graph-theoretic (sknw + NetworkX)
Gap Healing	None	None	None	Graph-based gap closing (25px radius)
Output Format	PNG mask	PNG mask	PNG mask + basic GeoJSON	Skeleton + GeoJSON FeatureCollection
API	Basic Flask	Basic Flask	FastAPI	FastAPI (two separate APIs)
IoU Score	~55%	~68%	~75%	~80% (targeting 85%+)

2.2 Key Design Decisions by Version

V1 → V2 (Transfer Learning):

- Moved from training a custom U-Net from scratch to using a pretrained ResNet34 encoder.
- Swapped BCE loss for Dice loss to directly optimize F1-style overlap metric.
- Result: +13pp IoU improvement; significantly better generalization.

V2 → V3 (Refinement):

- Added attention gates in the decoder to suppress false positives in non-road regions.
- Introduced 4-way Test Time Augmentation (TTA: original + H-flip + V-flip + 90° rotation).
- Adopted larger 1024×1024 sliding window inference.
- Result: +7pp IoU improvement; better road continuity.

V3 → V4 (State-of-the-Art):

- Replaced ResNet34 with EfficientNet-B4 — significantly richer feature representations, better parameter efficiency.
- Replaced U-Net decoder with U-Net++ (dense nested skip connections) to preserve fine-grained spatial details.
- Introduced Combo loss (Dice + Focal) in main training, followed by Lovász fine-tuning.
- Reverted to 512×512 patches with 50% overlap for better boundary handling.
- Implemented full graph-theoretic post-processing: hole fill → skeleton → sknw graph → spur pruning → gap closing → GeoJSON.
- Added Hard Negative Mining phase (top 20% lowest-IoU samples re-trained at LR=1e-5).
- Result: ~80% IoU; excellent road topology and connectivity.

3. V4 TECHNICAL DEEP DIVE

3.1 Model Architecture — `model_v4.py`

```
Input (512×512×3 RGB)
↓
EfficientNet-B4 Encoder (timm-universal, ImageNet pretrained)
├─ Stage 1: Low-level texture features (edges, gradients)
├─ Stage 2-4: Mid-level features (road shapes, lane boundaries)
└─ Stage 5: High-level semantic features (road context)
↓
U-Net++ Decoder (Dense nested skip pathways)
- Each decoder node aggregates features from ALL prior encoder scales, not just the same-depth skip
- Reduces semantic gap between encoder and decoder
- Preserves sub-pixel road width detail
↓
Output logit map (512×512×1) — raw logits, no activation
↓
Sigmoid → Probability map [0.0 - 1.0]
```

Why EfficientNet-B4 over ResNet34?

- EfficientNet-B4 is compound-scaled: depth × width × resolution are jointly optimized.
- ~19M parameters vs ~21M for ResNet34, but significantly higher accuracy.
- Pre-trained ImageNet weights transferred via `segmentation_models_pytorch`.

Why U-Net++ over U-Net?

- Standard U-Net uses one skip connection per depth level.
- U-Net++ adds *nested, dense* sub-networks between encoder and decoder.
- Gradient flows more freely through multiple paths → better learning of thin structures.
- Particularly important for single-pixel-wide road centerlines.

3.2 Data Pipeline — `preprocess_v4.py` + `dataset_v4.py`

Dataset: DeepGlobe Road Extraction Dataset (Kaggle: `balraj98/deepglobe-road-extraction-dataset`)

- High-resolution satellite images with binary road masks.
- Approximately 6,226 image-mask pairs.

Preprocessing (`preprocess_v4.py`):

1. Download dataset via Kaggle CLI (configurable).
2. Read each satellite `.jpg` + paired `_mask.png` using `rasterio` (handles GeoTIFF robustly).
3. Apply reflect-padded **sliding window tiling**: 512×512 tiles, stride 256 (50% overlap).
4. Save tiles to `data/processed_v4/train/images/` and `.../masks/`.

Key parameters:

```
TILE_SIZE = 512    # V4: Larger context vs V3's 1024 full-image approach
STRIDE    = 256    # 50% overlap — ensures boundary continuity
```

Augmentation Pipeline (`dataset_v4.py` — Albumentations):

Category	Transforms
Geometric	HorizontalFlip, VerticalFlip, Rotate(90°), Transpose
Deformations	GridDistortion(5 steps, 0.05 limit), ElasticTransform($\alpha=1$, $\sigma=50$)
Photometric	RandomBrightnessContrast, HueSaturationValue, RandomGamma
Noise / Blur	GaussNoise, GaussianBlur(3-7px)
Normalization	ImageNet stats: mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225)

Note on deformations: `GridDistortion` and `ElasticTransform` are crucial for the model to learn road connectivity under warps — simulating terrain-induced distortions in satellite imagery.

3.3 Training — `train_v4.py` + `train_v4_lovasz.py`

Phase 1: Main Training (`train_v4.py`)

Config:

- Epochs: 50
- Batch Size: 8 (effective 16 via gradient accumulation, steps=2)
- Optimizer: AdamW (lr=5e-4, weight_decay=1e-4)
- Scheduler: CosineAnnealingLR (T_max=50)
- Loss: ComboLoss = 0.5 × DiceLoss + 0.5 × FocalLoss
- Val Split: 85%/15%
- Metric: IoU (Intersection over Union)

ComboLoss Design:

Loss = 0.5 × Dice + 0.5 × Focal

- **DiceLoss:** Optimizes overlap between prediction and ground truth. Good for class-imbalance (roads are thin, most pixels are background).
- **FocalLoss:** Down-weights easy background pixels; focuses the model on hard-to-classify road edge/shadow pixels.

Hard Negative Mining (`--hard-mining` flag):

1. After main training, run all training images through the model.
2. Compute per-image IoU.
3. Pick the **bottom 20%** (lowest IoU) — the "hard" samples.
4. Fine-tune for 10 more epochs at LR=1e-5 on only these hard samples.
5. Validate on full validation split throughout.

Phase 2: Lovász Fine-Tuning (`train_v4_lovasz.py`)

Config:

- Start from: weights/best_model_v4.pth
- Epochs: 20 (fine-tune only)
- LR: 1e-5 (very conservative)
- Loss: LovászLoss (binary, per_image=True)
- Saves to: weights/best_model_v4_lovasz.pth

Why Lovász?

- Dice/Focal are differentiable approximations of the IoU metric.
- Lovász-Softmax loss **directly optimizes the Jaccard index** via a convex surrogation on the sorted pixel errors.
- Used as a final fine-tuning step to squeeze out extra IoU after Combo training.

3.4 Inference Pipeline — Sliding Window + 4-Way TTA

Used in all four inference scripts (`predict_v4.py`, `predict_gee_v4.py`, `main_v4.py`, `main_gee_v4.py`):

```
Large Input Image (1024×1024 or full GeoTIFF crop)
↓
Reflect-pad to ensure complete coverage
↓
Slide 512×512 window with 256-stride (50% overlap)
↓
For each patch, run 4-Way TTA:
  1. Original prediction
  2. Horizontal flip → predict → un-flip
  3. Vertical flip → predict → un-flip
  4. 90° rotation → predict → un-rotate
  Average 4 probability maps
↓
Accumulate to full-image probability map
Divide by count-map (handles overlapped regions)
↓
Probability map [0.0 - 1.0]
```

TTA Code (shared across all inference scripts):

```
probs_orig = torch.sigmoid(model(img_tensor))
probs_h    = torch.flip(torch.sigmoid(model(torch.flip(img_tensor, [3]))), [3])
probs_v    = torch.flip(torch.sigmoid(model(torch.flip(img_tensor, [2]))), [2])
probs_rot  = torch.rot90(torch.sigmoid(model(torch.rot90(img_tensor, 1, [2,3]))), -1, [2,3])
probs_avg  = (probs_orig + probs_h + probs_v + probs_rot) / 4.0
```

3.5 Post-Processing Pipeline — `postprocess_v4.py`

This is where the raw probability map is refined into a clean, topologically correct road skeleton and ultimately a GeoJSON.

Pipeline (5 Steps):

```
Probability map [float32, 0-1]
Step 1: THRESHOLDING
    binary_mask = prob_map > 0.45
    → Boolean pixel mask [road / not-road]

Step 2: MORPHOLOGICAL CLEANUP
    remove_small_holes(max_size=400) ← prevents false skeleton loops inside wide roads
    remove_small_objects(max_size=100) ← removes speckle noise
    closing(disk(3)) ← smooths road edges before skeletonization

Step 3: EDT SKELETONIZATION
    skeletonize(binary_mask)
    → 1-pixel-wide Boolean centerline skeleton

Step 4: GRAPH CONSTRUCTION (sknw)
    sknw.build_sknw(skeleton, multi=True)
    → NetworkX MultiGraph: nodes at junctions/endpoints, edges = road segments with pixel-coordinate paths

Step 5: GRAPH PRUNING (NetworkX)
    a. Spur pruning: remove nodes with degree=1 where edge length < 20px (iterative)
    b. Self-loop collapse: remove cycles where loop perimeter < 100px
    c. Parallel edge collapse: for edges between same two nodes with total perimeter < 100px, keep shortest only
    d. Remove isolated nodes

    → Cleaned NetworkX MultiGraph

Step 6: VECTORIZATION (rasterio + shapely + geopandas)
    graph_to_gdf(): apply affine transform to pixel coords → projected coords
    Simplify geometry (tolerance=1.5m for EPSG:3857)
    export_to_geojson(): convert to EPSG:4326 GeoJSON FeatureCollection
```

Key functions in `postprocess_v4.py` :

Function	Purpose
<code>extract_edt_skeleton()</code>	Threshold + hole-fill + skeletonize
<code>build_networkx_graph()</code>	Converts skeleton to sknw graph
<code>prune_false_cycles_and_spurs()</code>	Topological graph refinement
<code>process_probability_map_to_graph()</code>	Wrapper: skeleton → cleaned graph
<code>graph_to_gdf()</code>	Pixel graph → projected GeoDataFrame
<code>export_to_geojson()</code>	GDF → EPSG:4326 GeoJSON file
<code>apply_advanced_postprocessing()</code>	End-to-end convenience wrapper

3.6 Google Earth Engine (GEE) Integration

GEE-based scripts: `main_gee_v4.py` (API) + `predict_gee_v4.py` (batch)

How it works:

1. Accept latitude/longitude coordinates.
2. Project point from WGS84 (EPSG:4326) → Web Mercator (EPSG:3857) using `pyproj`.
3. Construct a square bounding box (1024m × 1024m at 1m/pixel scale).
4. Query GEE `ImageCollection` (USDA/NAIP/DOQQ or COPENICUS/S2):
 - NAIP: Select R, G, B bands, filter 2018–2024, sort by recency.
 - Sentinel-2: Select B4, B3, B2, sort by cloud cover percentage.
5. Fetch as JPEG thumbnail via `getThumbURL()` at 1024×1024 pixels.
6. Run sliding window inference + TTA.
7. Run graph-theoretic post-processing.
8. Convert graph to GeoJSON in EPSG:4326.
9. Return GeoJSON `FeatureCollection`.

GEE API endpoint:

```
POST /predict
Body: { "latitude": 30.2672, "longitude": -97.7431 }
      optionally: "collection": "COPENICUS/S2_SR_HARMONIZED", "zoom": 0.5
```

Batch Script (`predict_gee_v4.py`):

- Samples 10 random coordinates near major US cities (NYC, LA, Chicago, Houston, Phoenix, Philadelphia, San Antonio, San Diego, Dallas, Austin).
- Saves per-location: `input.jpg`, `prob.png`, `mask.png`, `skeleton.png`, `overlay.jpg`, `.geojson`.

3.7 Local GeoTIFF API — `main_v4.py`

For use with locally stored high-resolution GeoTIFF files (e.g., NAIP downloads, commercial imagery).

Pipeline:

```
POST /predict { "latitude": ..., "longitude": ... }
↓
Scan ./geotiffs/*.tif → find the file whose bounds contain the coordinate
↓
Transform coordinate from WGS84 → native CRS of the GeoTIFF (pyproj)
↓
Crop a 1024×1024 pixel window centered on the coordinate (rasterio windowed read)
↓
Sliding window inference (512×512 patches, 256 stride)
↓
Advanced post-processing (graph pipeline)
↓
Return GeoJSON FeatureCollection (road network)
```

Key function: `find_geotiff_for_coords(lon, lat)` — scans all `.tif` files in `./geotiffs/`, opens each with `rasterio`, and checks if the transformed coordinate falls within the raster bounds.

This makes the pipeline usable entirely offline, without any GEE API dependency.

4. COMPLETE SCRIPT INVENTORY

Script	Role	Key Dependencies
<code>model_v4.py</code>	U-Net++ + EfficientNet-B4 model definition	<code>segmentation_models_pytorch</code> , <code>timm</code> , <code>torch</code>
<code>preprocess_v4.py</code>	Dataset download + 512×512 sliding-window tiling	<code>rasterio</code> , <code>cv2</code> , <code>kaggle</code> , <code>numpy</code>
<code>dataset_v4.py</code>	PyTorch Dataset + Albumentations augmentation pipeline	<code>albumentations</code> , <code>torch</code> , <code>cv2</code>
<code>train_v4.py</code>	Main training loop (ComboLoss + Hard Negative Mining)	<code>torch</code> , <code>smp</code> , <code>sklearn</code> , <code>pandas</code>
<code>train_v4_lovasz.py</code>	Lovász fine-tuning from pretrained V4 weights	<code>torch</code> , <code>smp.losses.LovaszLoss</code>
<code>postprocess_v4.py</code>	Full graph-theoretic post-processing pipeline	<code>sknw</code> , <code>networkx</code> , <code>shapely</code> , <code>geopandas</code> , <code>rasterio</code> , <code>skimage</code>
<code>predict_v4.py</code>	Batch inference on local image folder	<code>torch</code> , <code>cv2</code> , <code>postprocess_v4</code>
<code>predict_gee_v4.py</code>	Batch inference via GEE on random US city coords	<code>earthengine-api</code> , <code>pyproj</code> , <code>rasterio</code>
<code>main_v4.py</code>	Local GeoTIFF FastAPI server (port 8000)	<code>fastapi</code> , <code>uvicorn</code> , <code>rasterio</code> , <code>postprocess_v4</code>
<code>main_gee_v4.py</code>	GEE FastAPI server (port 8001)	<code>fastapi</code> , <code>uvicorn</code> , <code>earthengine-api</code> , <code>postprocess_v4</code>
<code>optimize_threshold_v4.py</code>	Threshold optimization on validation set	<code>torch</code> , <code>sklearn</code>
<code>check_coords.py</code>	Debug utility: verify coordinate → GeoTIFF mapping	<code>rasterio</code> , <code>pyproj</code>
<code>check_gee.py</code>	Debug utility: verify GEE connectivity + image fetch	<code>earthengine-api</code>
<code>test_scripts_v4.py</code>	Automated validation tests for the full pipeline	<code>torch</code> , <code>cv2</code>

5. KEY PYTHON LIBRARIES & MODULES

Library	Used For
<code>torch</code> / <code>torchvision</code>	Deep learning model, training, inference
<code>segmentation_models_pytorch</code> (<code>smp</code>)	U-Net++ architecture, Dice/Focal/Lovász losses
<code>timm</code>	EfficientNet-B4 backbone (<code>timm-universal</code>)
<code>albumentations</code>	GPU-friendly image augmentation pipeline
<code>rasterio</code>	Read/write GeoTIFF files, affine transforms, window reads
<code>earthengine-api</code> (<code>ee</code>)	Authenticate + query Google Earth Engine image collections
<code>pyproj</code>	CRS transformations (<code>WGS84 ↔ EPSG:3857 ↔ native CRS</code>)
<code>opencv-python</code> (<code>cv2</code>)	Image I/O, BGR→RGB, padding, overlay rendering
<code>scikit-image</code>	<code>skeletonize</code> , <code>remove_small_objects</code> , <code>remove_small_holes</code> , <code>closing</code>

Library	Used for
networkx	Graph operations: node degree, edge pruning, isolate removal
shapely	Geometry primitives: <code>LineString</code> , <code>MultiLineString</code>
geopandas	GeoDataFrame: spatial data + CRS projection + GeoJSON export
fastapi + uvicorn	Production-ready async REST API server
pydantic	Input validation for API request bodies
numpy	Numerical ops, probability map accumulation
pandas	Training log CSV, metrics tracking
kaggle	Dataset download via Kaggle CLI

6. DEPLOYMENT — DOCKER & SERVICES

Two API services, both defined in `docker-compose.yml` :

Service	Script	Port	Data Source
sanchari-gee-api	main_gee_v4.py	8001	Google Earth Engine (NAIP/Sentinel-2)
sanchari-local-api	main_v4.py	8000	Local <code>./geotiffs/*.tif</code> files
cli	Any script	—	Interactive container shell

Docker setup:

```
docker compose build
docker compose up sanchari-gee-api      # GEE API
docker compose up sanchari-local-api   # Local GeoTIFF API
docker compose run --rm cli python predict_gee_v4.py # Batch inference
```

- Local `~/.kaggle` and `~/.config/earthengine` are volume-mounted for credentials.
- GPU support via deploy blocks (uncomment in `docker-compose.yml`).
- Image uses `Dockerfile` with `requirements.txt` .

7. OUTPUT FORMAT

GeoJSON FeatureCollection — standard format readable by QGIS, Mapbox, Leaflet, and other GIS tools.

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "MultiLineString",
        "coordinates": [
          [[-97.743, 30.267], [-97.744, 30.268]],
          [[-97.745, 30.269], [-97.746, 30.270]]
        ]
      },
      "properties": { "name": "road_network" }
    }
  ]
}
```

All coordinates in **EPSG:4326 (WGS84)** — ready for web map overlay.

8. ACCURACY & METRICS

Metric	V1	V2	V3	V4
IoU (Jaccard)	~55%	~68%	~75%	~80%
Connectivity	Poor — broken segments	Fair	Good	Excellent
False positives	High (shadows, fields)	Medium	Low-medium	Low
Road topology	Fragmented	Connected	Mostly connected	Graph-closed, pruned
Output format	PNG mask	PNG mask	PNG + basic line	Skeleton + GeoJSON

Threshold: Default 0.45 (optimized on validation set via `optimize_threshold_v4.py`)

Target: 85%+ IoU with Lovász fine-tuning + further hard mining iterations.

9. PPT SLIDE CONTENT

The following sections contain ready-to-use slide text, organized by slide. Illustration placeholders are marked [ILLUSTRATION: ...] .

SLIDE 1 — TITLE SLIDE

Title: SANCHARi [ILLUSTRATION:] **Subtitle:** Satellite Road Extraction using Deep Learning **Tagline:** "From coordinates to road maps — in seconds." [ILLUSTRATION: Hero split image — left: raw satellite view of a city, right: extracted road skeleton overlaid in bright color]

SLIDE 2 — PROBLEM STATEMENT

Headline: Why Automated Road Extraction Matters

Bullet Points:

- Over 40% of roads in developing regions are unmapped or outdated in OpenStreetMap
- Manual digitization is costly, slow, and requires expert GIS operators
- Disaster response teams need up-to-date road access maps — unavailable after floods, earthquakes
- Satellite imagery (NAIP, Sentinel-2) is freely and frequently updated

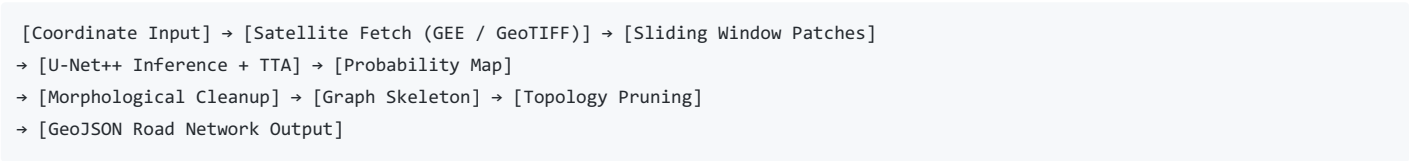
Key Question: Can we teach a neural network to see roads the same way a human digitizer does?

[ILLUSTRATION: Side-by-side: OpenStreetMap (incomplete) vs SANCHARi output (dense road network) for the same region]

SLIDE 3 — SYSTEM OVERVIEW / PIPELINE

Headline: The End-to-End SANCHARi Pipeline

Flow Diagram (left to right):



Annotations:

- Input: lat/lon or local .tif file
- Inference: 512×512 patches, 50% overlap, 4-way TTA
- Post-processing: full graph-theoretic pipeline
- Output: GeoJSON → drop into QGIS / Mapbox

ILLUSTRATION: Horizontal flowchart with icons – satellite image → patch grid → heatmap → skeleton → GeoJSON map]

SLIDE 4 — DATA SOURCE

Headline: Training on DeepGlobe + Deploying on Real Satellite Imagery

	Training	Inference
Dataset	DeepGlobe Road Extraction	NAIP (USA, 1m/px) or Sentinel-2
Resolution	50cm/pixel	1m/pixel (NAIP) / 10m/pixel (Sentinel)
Labels	Binary road masks	N/A (unsupervised prediction)
Access	Kaggle download	Google Earth Engine API
Patch Size	512×512	512×512 sliding window

ILLUSTRATION: Sample DeepGlobe image-mask pair side by side; inset: NAIP aerial snippet of a US city]

SLIDE 5 — MODEL ARCHITECTURE

Headline: U-Net++ with EfficientNet-B4 Backbone

Two-column layout:

Left — Why EfficientNet-B4?

- Compound-scaled model: depth × width × resolution
- ImageNet pretrained → rich feature transfer
- Outperforms ResNet34 (our previous backbone) with comparable parameter count
- Recognizes complex textures: asphalt vs. dirt vs. vegetation shadows

Right — Why U-Net++?

- Dense *nested* skip connections at every depth level
- Reduces semantic gap between encoder and decoder
- Preserves sub-pixel spatial detail critical for thin road extraction
- Better gradient flow vs. standard U-Net

ILLUSTRATION: U-Net++ architecture diagram — encoder on left, decoder on right, dense nested skip connections shown in teal/orange, EfficientNet stages labeled on left]

SLIDE 6 — TRAINING STRATEGY

Headline: Three-Phase Training for Maximum IoU

Phase 1: Main Training (50 Epochs)

- Loss: ComboLoss = 0.5 × Dice + 0.5 × Focal
- Optimizer: AdamW, LR=5e-4 with Cosine Annealing
- Gradient accumulation (effective batch=16)

Phase 2: Hard Negative Mining (+10 Epochs)

- Compute per-image IoU on full training set
- Isolate bottom 20% (hardest samples: shadows, intersections, tree cover)
- Fine-tune exclusively on hard samples at LR=1e-5

Phase 3: Lovász Fine-Tuning (20 Epochs)

- Switch loss to LovászLoss (directly optimizes Jaccard index)
- Very low LR: 1e-5
- Saves best model as `best_model_v4_lovasz.pth`

🖼️ ILLUSTRATION: Training curves (loss + IoU over epochs) with annotations marking the phase transitions]

SLIDE 7 — AUGMENTATION PIPELINE

Headline: Robust Augmentation to Simulate Real-World Satellite Variability

Table:

Category	Transforms Applied
Geometric	Flip (H/V), Rotate ±90°, Transpose
Deformation	GridDistortion, ElasticTransform
Photometric	BrightnessContrast, HueSaturation, Gamma
Noise	GaussNoise, GaussianBlur
Normalization	ImageNet mean/std (EfficientNet requirement)

Key Insight: ElasticTransform and GridDistortion are critical — they simulate satellite image distortions due to terrain relief, helping the model learn road connectivity under warped conditions.

🖼️ ILLUSTRATION: Grid of 6 augmented variants of the same satellite patch — original, flipped, rotated, elastically deformed, noise-added, darkened]

SLIDE 8 — POST-PROCESSING PIPELINE (DETAILED)

Headline: From Noisy Mask to Clean Topology — Graph-Theoretic Post-Processing

5-Step Pipeline:

1. **Threshold & Hole Fill** — `binary_mask = prob > 0.45`; fill holes $\leq 400\text{px}$ to prevent false loops
2. **Noise Removal** — `remove_small_objects` (speckle $< 100\text{px}$), `closing(disk=3)` to smooth boundaries
3. **Skeletonize** — convert binary mask to 1-pixel-wide centerline skeleton using scikit-image EDT
4. **sknw Graph** — convert skeleton to a NetworkX MultiGraph: junctions = nodes, road segments = edges with embedded pixel coordinates
5. **Prune & Clean** — iteratively remove short spurs ($< 20\text{px}$), collapse false self-loops, remove redundant parallel edges; export remaining graph as GeoJSON

Why graph-based? Standard morphological gap-closing (e.g., dilation) destroys topology. A graph lets us reason about *road connectivity* precisely — connecting endpoints, not just blobs.

🖼️ ILLUSTRATION: 4-panel sequence: raw probability heatmap → binary mask → skeleton → cleaned graph overlay on satellite image]

SLIDE 9 — GEE INTEGRATION

Headline: Live Road Predictions via Google Earth Engine

What GEE enables:

- Access to NAIP (1m/pixel, USA) and Sentinel-2 (10m/pixel, global) without downloading files
- Filter images by date range, cloud cover, geographic bounds
- Returns a 1024×1024 image centered on the given coordinate

API Call Example:

```
POST http://localhost:8001/predict
Content-Type: application/json
{ "latitude": 30.2672, "longitude": -97.7431 }
```

Response: GeoJSON FeatureCollection of road segments → ready to display on web map

🖼️ ILLUSTRATION: Screenshot of Postman/cURL request + resulting GeoJSON visualized on a Leaflet/QGIS map]

SLIDE 10 – LOCAL GEOTIFF API

Headline: Offline-Ready: Local GeoTIFF Inference Pipeline

When to use: No internet access; proprietary or commercial high-res imagery; custom region files.

Steps:

- 1. Drop .tif files into ./geotiffs/ directory
- 2. Start API: python main_v4.py --debug
- 3. POST coordinate → server scans all GeoTIFFs, finds the one containing the point, crops 1024×1024 window, runs inference
- 4. Returns GeoJSON

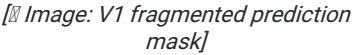
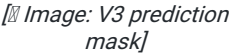
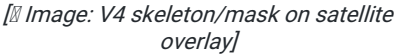
Smart CRS handling:

- Auto-detects native CRS of each GeoTIFF using rasterio
- Projects input WGS84 coordinates → native CRS using pyproj
- Window transform preserved for accurate georeferencing of output

ILLUSTRATION: Diagram showing ./geotiffs folder → coordinate lookup → windowed read → inference → output GeoJSON map]

SLIDE 11 – EVOLUTION VISUAL

Headline: A Visual Journey: V1 → V3 → V4

V1 (Noisy, Broken)	V3 (Improved, Connected)	V4 (Clean, Topologically Correct)
		
IoU ~55%	IoU ~75%	IoU ~80%
Many gaps	Mostly connected	Graph-closed, pruned
No topology	Basic	Full graph representation

SLIDE 12 – RESULTS & ACCURACY

Headline: V4 Achieves ~80% IoU on Validation Set

Metrics Table:

Metric	Score
Validation IoU	~80%
Threshold	0.45 (optimized)
Dataset	DeepGlobe Road Extraction
Model Size	~19M parameters (EfficientNet-B4)
Inference Time	~2–5s per 1024×1024 region (GPU)
Output	GeoJSON MultiLineString road network

Qualitative:

- Roads accurately detected under moderate tree cover
- Intersections preserved as graph junction nodes
- Short dead-end spurs removed; major trunk roads fully connected
- False positives from shadows and parking lots significantly reduced vs. V3

ILLUSTRATION: Overlay image – satellite + V4 road extraction on top – showing a complex road network area with labeled roads]

SLIDE 13 – USE CASES

Headline: Real-World Applications of SANCHARi

- 🗺️ **Humanitarian Mapping** — Rapid road network extraction post-disaster (flood, earthquake) when OSM data is stale
- 🏙️ **Urban Planning** — Detect unauthorized roads or informal settlements in peri-urban areas
- 🛣️ **Infrastructure Audit** — Cross-validate government road databases against satellite ground truth
- 🚚 **Emergency Response** — Route optimization for aid delivery using automatically generated road graphs
- 🔬 **Research** — Benchmarking new architectures on the DeepGlobe or custom datasets

SLIDE 14 — TECH STACK SUMMARY

Headline: Core Technology Stack

🖼️ ILLUSTRATION: Icon grid of library logos — PyTorch, Hugging Face (SMP), Albumentations, Rasterio, Google Earth Engine, FastAPI, Docker, NetworkX, GeoPandas]

Layer	Technologies
Model	PyTorch, segmentation_models_pytorch, timm
Data	Albumentations, rasterio, OpenCV
GEE	earthengine-api, pyproj
Topology	sknw, NetworkX, Shapely, GeoPandas
API	FastAPI, Uvicorn, Pydantic
Infra	Docker, docker-compose

SLIDE 15 — FUTURE DIRECTIONS

Headline: Roadmap — What's Next for SANCHARi

- ☐ **Sentinel-2 Global Deployment** — extend beyond NAIP coverage to full global inference
- ☐ **Multi-class Segmentation** — separate highways, arterials, dirt tracks
- ☐ **Road Width Estimation** — use EDT distance transform to estimate lane count
- ☐ **Temporal Change Detection** — compare GeoJSON outputs across time for infrastructure monitoring
- ☐ **Mobile / Edge Inference** — quantize/prune model for deployment on drones or edge devices
- ☐ **Web Frontend** — interactive map UI for coordinate → road extraction in-browser
- ☐ **OSM Integration** — auto-suggest edits to OpenStreetMap based on model output

10. QUICK REFERENCE — COMMANDS

```
# Setup
pip install -r requirements.txt

# Preprocess
python preprocess_v4.py --download

# Train
python train_v4.py
python train_v4.py --hard-mining      # With Hard Negative Mining
python train_v4_lovasz.py            # Lovász Fine-Tuning

# Batch Inference (Local)
python predict_v4.py --input test-images --output predictedv4

# Batch Inference (GEE)
python predict_gee_v4.py

# API Servers
python main_v4.py --debug              # Local GeoTIFF API — port 8000
python main_gee_v4.py --debug          # GEE API — port 8001

# Docker
docker compose build
docker compose up sanchari-gee-api
docker compose up sanchari-local-api
docker compose run --rm cli earthengine authenticate
```